

C Programming (W4)



Welcome!!

Please check attendance individually.
(Mobile App)

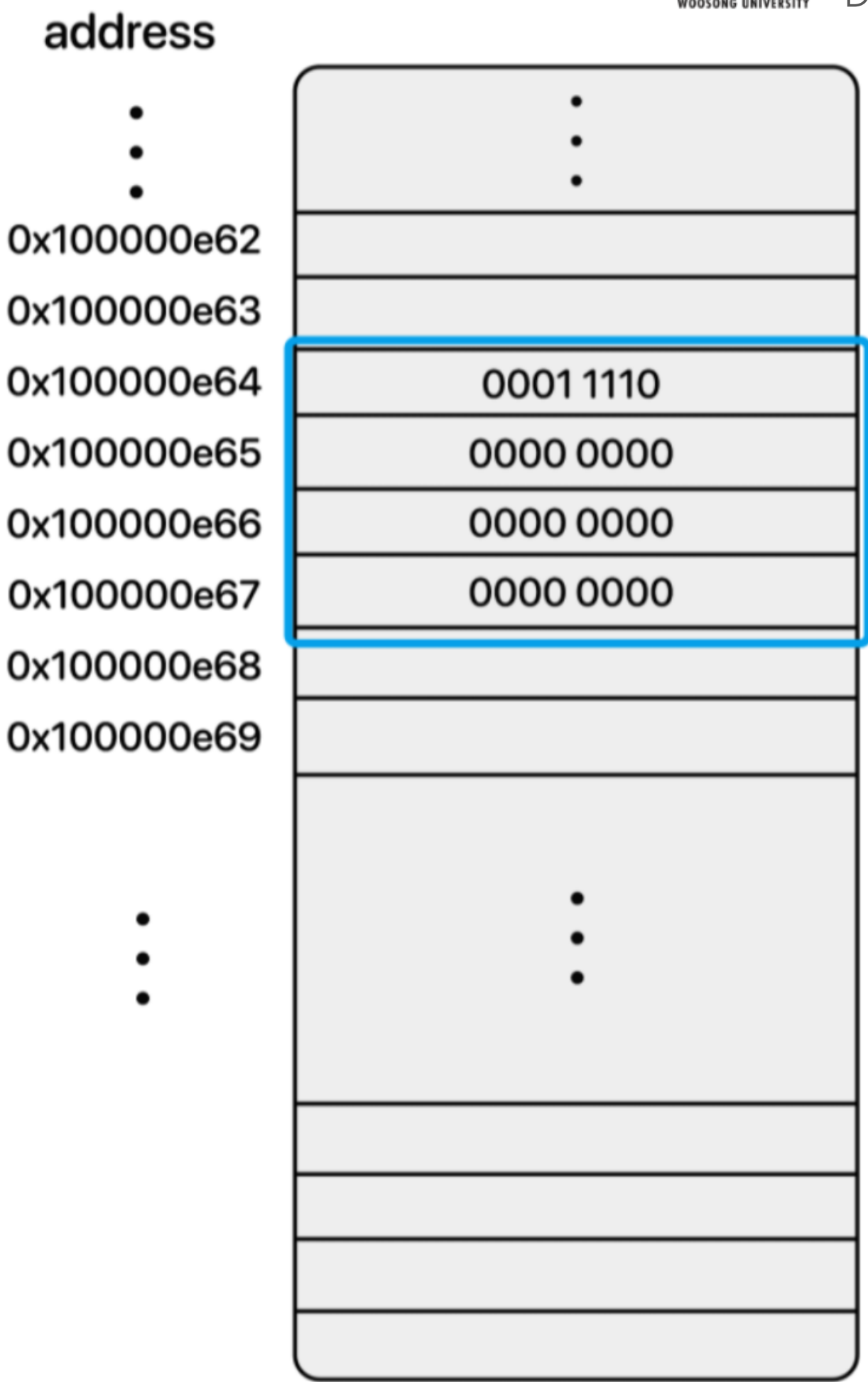
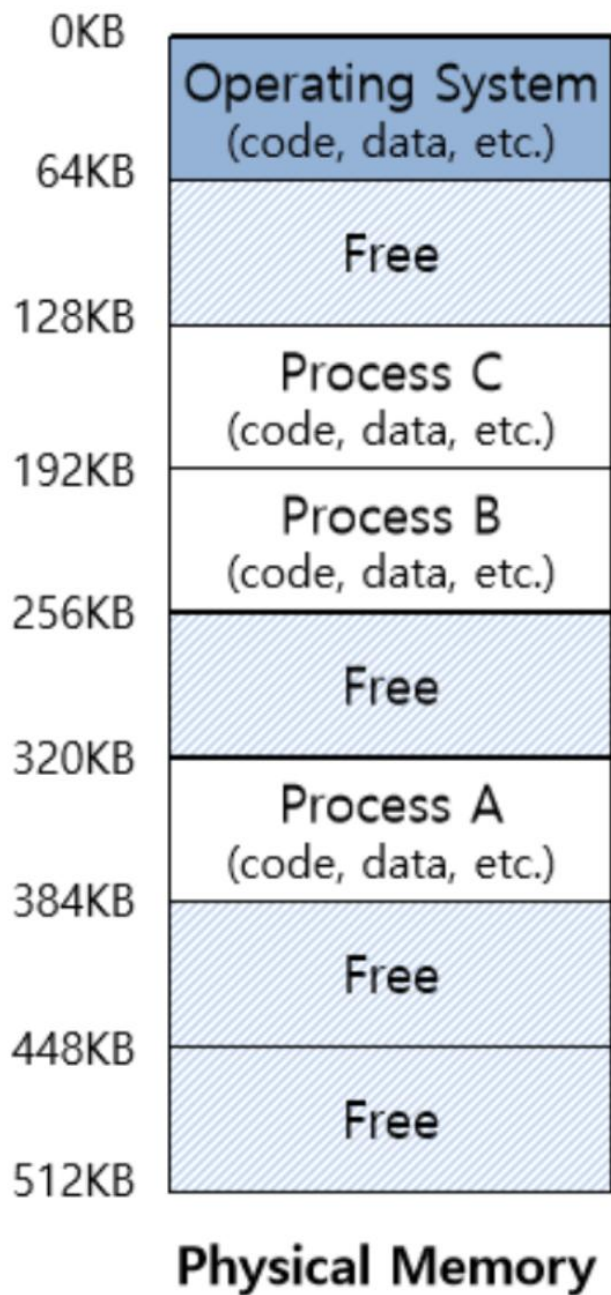
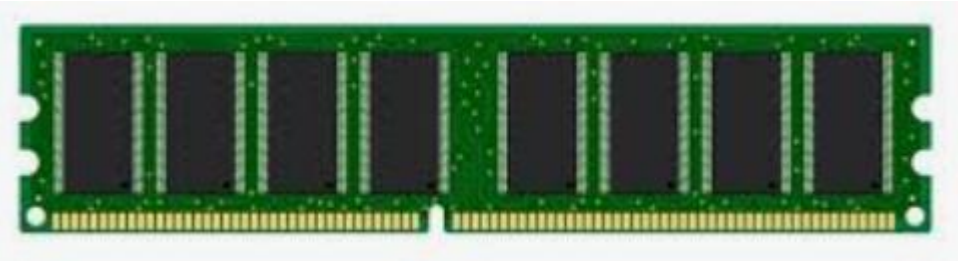
Things to do today

- 01** | Lecture Notes (Ch.3, 4)
- 02** | How to handle Language
- 03** | printf & scanf(ch.3) & Standard Input/Output

Memory

32 bit OS: Memory Address (4GB(2^32 byte)

64 bit OS: Memory Address (16EB(2^64 byte)



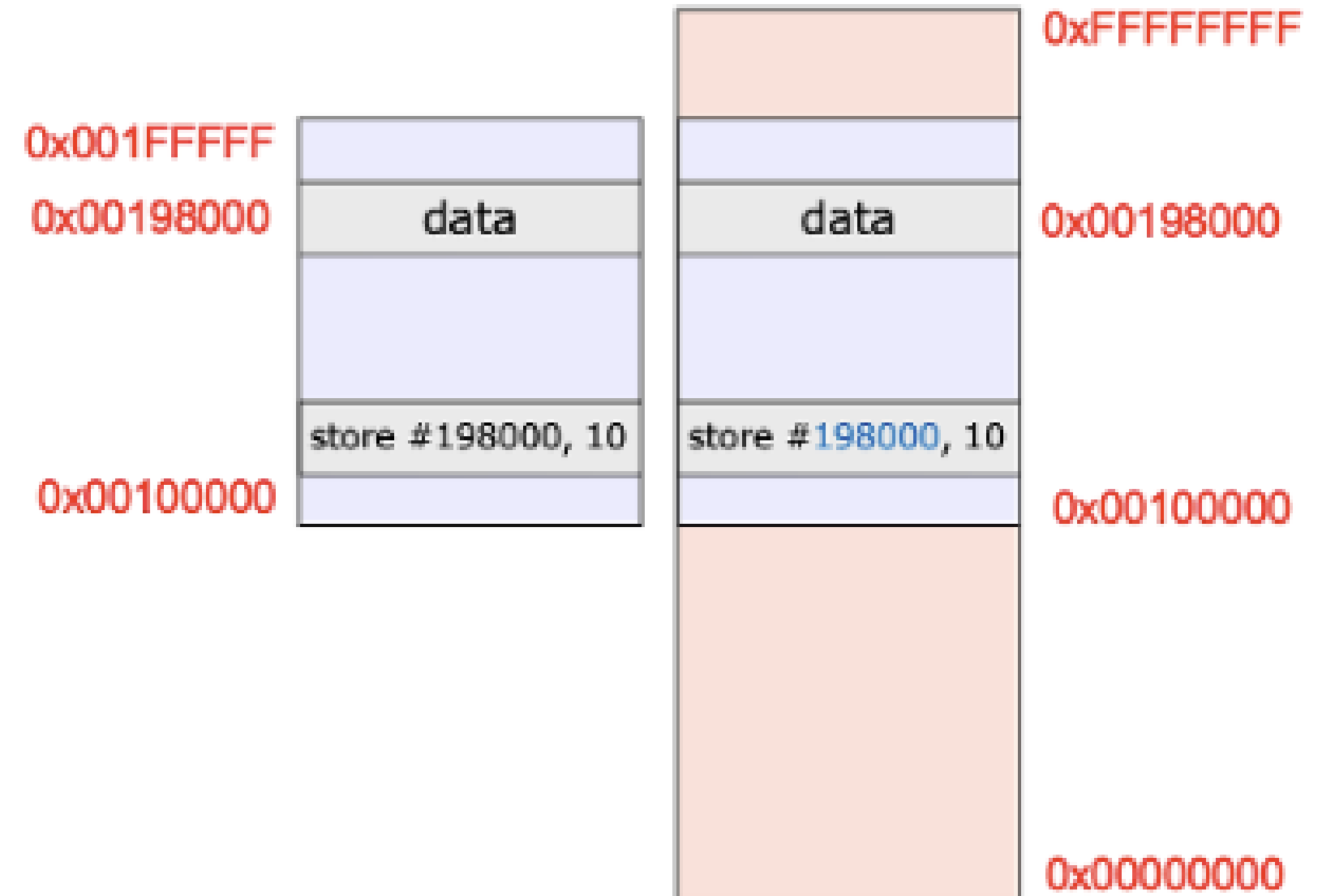
A d d r e s s e s	0xFFFFFFFF	1000 0000
		
	0x00000008	0100 1001
	0x00000007	1100 1100
	0x00000006	0110 1110
	0x00000005	0110 1110
	0x00000004	0000 0000
	0x00000003	0110 1011
	0x00000002	0101 0001
	0x00000001	1100 1001
	0x00000000	0100 1111

Main Memory

■ Compile time

```
void func( )
{
    int data;

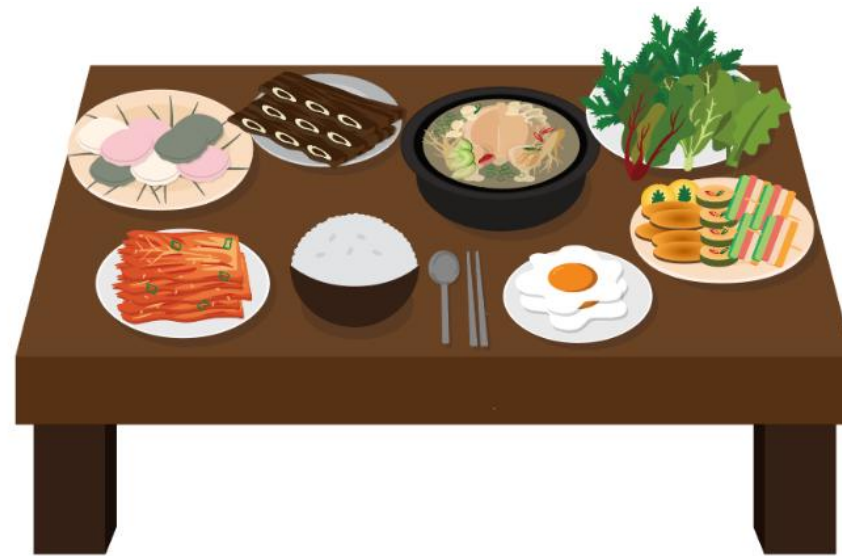
    ...
    data = 10;
    ...
}
```



A pointer represents an address

Variables & Data types

- Variable: A memory space where **data** can be stored. (bowl)
 - The bowl can hold rice, side dishes, and water.



- Variable creation and rules and features
 - Reserved words (keywords) cannot be used (for, if, else,...)
 - Spaces cannot be included
 - Only English letters and underscores (_) can be used as the first letter (number x)
 - Special characters other than underscores (_) cannot be used
 - Case sensitive

Variables & Data types

- Data types: To use memory space efficiently, data types of appropriate shape and size must be used.

Data Type	Description	Size (bytes)	Range	Example
int	Integer data type for whole numbers.	4	-2,147,483,648 to 2,147,483,647	int num = 10;
short	Short integer data type. Smaller range than int.	2	-32,768 to 32,767	short num = 100;
long	Long integer data type, typically used for larger numbers.	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long num = 1000000L;
long long	Extended long integer data type, used for even larger numbers.	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long long num = 1234567890123LL;
unsigned int	Unsigned integer, stores only positive values.	4	0 to 4,294,967,295	unsigned int num = 10U;
unsigned short	Unsigned short integer, stores only positive values.	2	0 to 65,535	unsigned short num = 100U;
unsigned long	Unsigned long integer, stores only positive values.	8	0 to 18,446,744,073,709,551,615	unsigned long num = 1000000UL;
unsigned long long	Unsigned long long integer, stores only positive values.	8	0 to 18,446,744,073,709,551,615	unsigned long long num = 1234567890123ULL;
char	Character data type, used to store single characters.	1	-128 to 127 (signed) or 0 to 255 (unsigned)	char letter = 'A';
unsigned char	Unsigned character, stores only positive character values (0 to 255).	1	0 to 255	unsigned char letter = 65U;
float	Single-precision floating point number.	4	$\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$	float num = 3.14f;
double	Double-precision floating point number, provides higher precision than float.	8	$\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$	double num = 3.141592;
long double	Extended precision floating point number (depends on the system).	10 or 16	Varies by system, typically $\pm 3.4 \times 10^{-4932}$ to $\pm 1.1 \times 10^{4932}$	long double num = 3.141592653589793;
_Bool	Boolean type (from C99 standard), stores true or false.	1	0 (false), 1 (true)	_Bool isTrue = 1;
void	Void type, used to indicate the absence of data or return type for functions.	N/A	N/A	void function() {}

Special Characters

- { } (Curly braces - Used for code blocks)
- [] (Square brackets - Used for arrays)
- () (Parentheses - Used for functions and expressions)
- ; (Semicolon - Statement terminator)
- : (Colon - Used in labels and ternary operator)
- # (Hash - Preprocessor directive)
- " (Double quotes - String literals)
- ' (Single quotes - Character literals)
- \ (Backslash - Escape sequences)
- // (Single-line comment)
- /* */ (Multi-line comment)
- ! (Exclamation mark) NOT operator

ASCII table

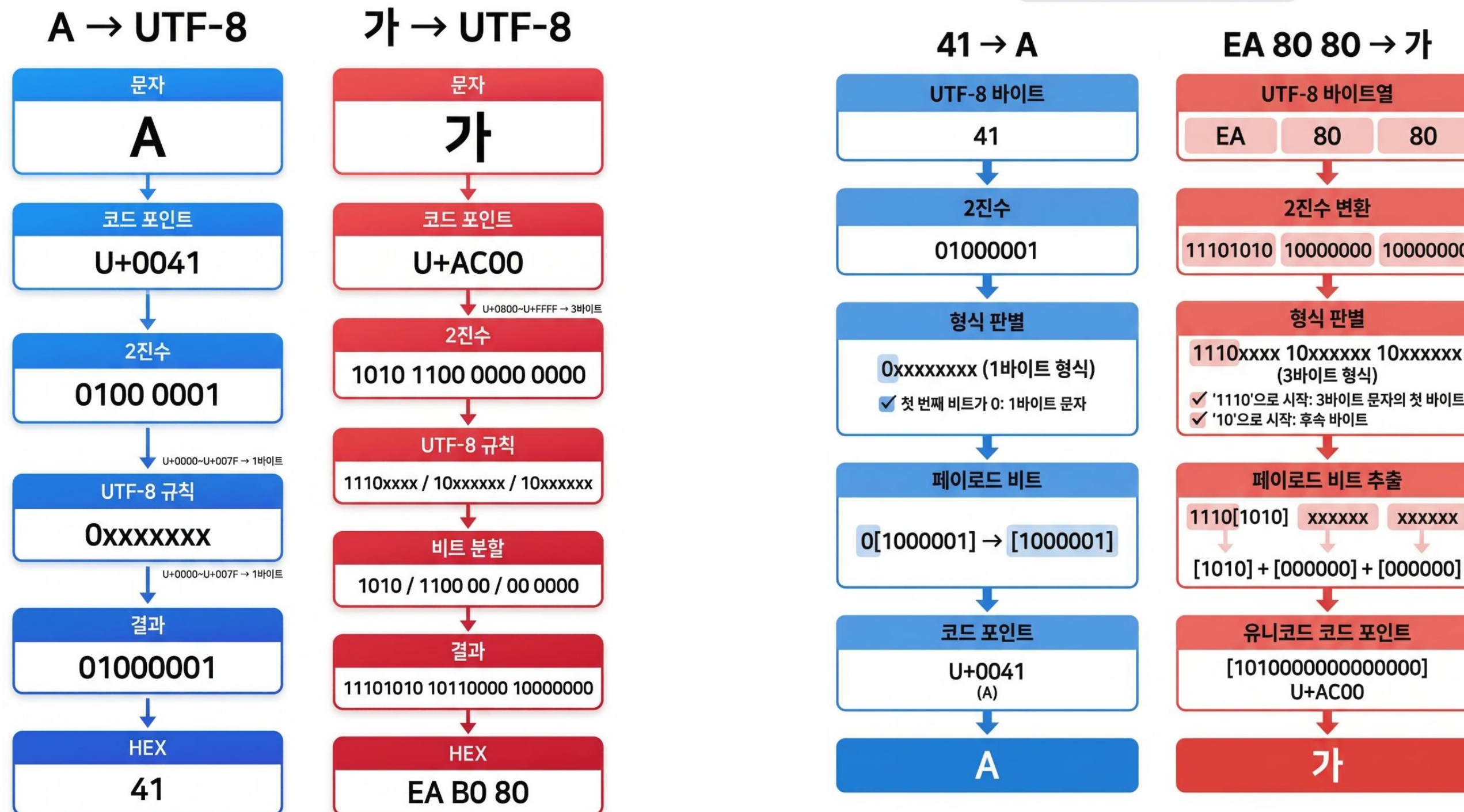
10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자
0	0x00	NUL	32	0x20	Space	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	TAB	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	\	124	0x7C	
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

문자 저장과 해석 방법 (인코딩 & 디코딩)

Character Storage and Interpretation Methods (Encoding & Decoding)

UTF-8 디코딩 과정

바이트열에서 유니코드 문자로 복원



Unicode

C was originally designed for **ASCII (1-byte characters)**, but modern applications require **Unicode** support to handle characters from multiple languages. Unicode works through different encoding formats such as **CP949(Windows)**, **UTF-8**, **UTF-16**, and **UTF-32**.

Unicode is a universal character set that assigns a unique number to every character in every language.

ex) 'A' → U+0041 (Just number)

'가' → U+AC00

'あ' → U+3042

Unicode defines the meaning of characters but not how they are stored in memory or files.

Range: U+0000 ~ U+10FFFF → a total of 1,114,112 possible code points.

Within this range, Unicode is designed to represent all the world's scripts, emojis, symbols, and even ancient characters.

Method of expression using UTF-8:

- U+0000 ~ U+007F → 1 byte
- U+0080 ~ U+07FF → 2 bytes
- U+0800 ~ U+FFFF → 3 bytes
- U+10000 ~ U+10FFFF → 4 bytes

Unicode Encoding in C

Encoding: a way to encode Unicode characters as bytes, so they can be stored or transmitted.

Unicode characters can be represented using:

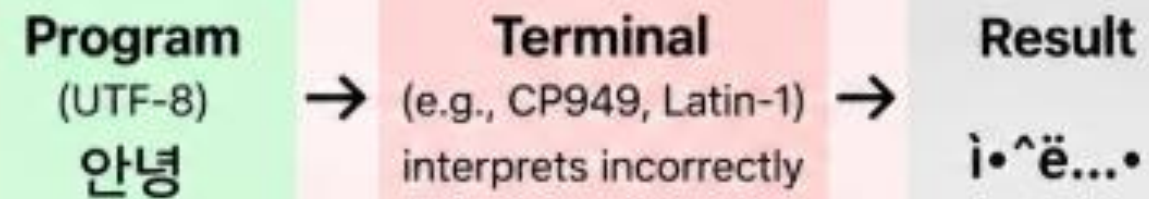
- **UTF-8** (variable-length, 1–4 bytes) → Most widely used (default in Linux/macOS)
 - ASCII character: 1 byte
 - European character: 2 bytes
 - Korean/Chinese etc: 3 ~ 4 bytes
- 1 byte: Basic ASCII characters (e.g. A, B, C, 0-9) → 0xxxxxxx
- 2 bytes: European languages, Korean initial consonants (e.g. é, ñ, ü, 한) → 110xxxxx 10xxxxxx
- 3 bytes: Most Korean characters, Hanja, some emojis → 1110xxxx 10xxxxxx 10xxxxxx
- 4 bytes: Extended characters and emojis (e.g. 😊, 🏆) → 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Why multilingual text looks broken in a terminal?

Usually because of encoding or font issues.

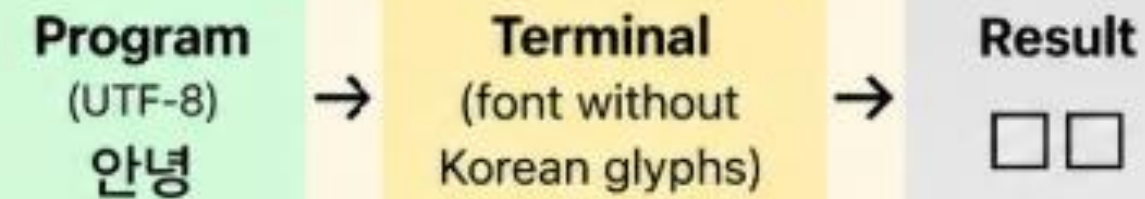
1 Encoding mismatch

The program outputs text in one encoding (e.g., UTF-8), but the terminal interprets it using a different encoding.



2 Font does not support the character

The encoding is correct, but the terminal's font does not have a glyph for the character.



How to fix (except #3: font-related)

1 Use the same encoding

- Make sure both the program and the terminal use the same encoding (usually UTF-8).
- If your program outputs a different encoding, change it to UTF-8 (e.g., compile or run with UTF-8 output).

2 Set UTF-8 code page in Windows (using `chcp`)

- In Windows cmd.exe, set the code page to UTF-8:

```
C:\> chcp 65001
Active code page: 65001
```

Now the terminal uses UTF-8.

In short: use the same encoding (e.g., `chcp 65001` on Windows).

Language

Code Page (chcp) Reference for the Listed Countries (Unique)

Use **chcp 65001** (UTF-8) for all countries when possible.

#	Country	Main language(s)	Primary script	Typical Windows code page (chcp)	Recommended chcp
1	 Bangladesh	Bengali (Bangla)	Bengali	57003 (ISCII Bangla)	65001
2	 Belgium	French, Dutch, German	Latin	1252 (Western European)	65001
3	 Nepal	Nepali	Devanagari	57002 (ISCII Devanagari)	65001
4	 Kazakhstan	Kazakh, Russian	Cyrillic	1251 (Cyrillic)	65001
5	 Indonesia	Indonesian	Latin	1252 (Western European)	65001
6	 Kenya	English, Swahili	Latin	1252 (Western European)	65001
7	 Zimbabwe	English, Shona, Ndebele	Latin	1252 (Western European)	65001
8	 Pakistan	Urdu, English	Arabic (Perso-Arabic)	1256 (Arabic)	65001
9	 Tunisia	Arabic, French	Arabic, Latin	1256 (Arabic) / 1252	65001
10	 Myanmar	Burmese	Myanmar	Not well supported (use UTF-8)	65001
11	 South Africa	English, Afrikaans, etc.	Latin	1252 (Western European)	65001
12	 Sri Lanka	Sinhala, Tamil	Sinhala, Tamil	Not well supported (use UTF-8)	65001
13	 Kyrgyzstan	Kyrgyz, Russian	Cyrillic	1251 (Cyrillic)	65001
14	 Tanzania	Swahili, English	Latin	1252 (Western European)	65001
15	 Azerbaijan	Azerbaijani	Latin (Azeri)	1254 (Turkish) (closest)	65001



Recommended (for all countries)

chcp 65001

- Uses UTF-8 (supports almost all languages)
- Works for multilingual text together



Notes

- chcp sets the terminal's code page.
- Your program must also output text in UTF-8.
- Some languages (e.g., Burmese, Sinhala) are not well supported by legacy code pages — use UTF-8.



Conclusion

For your list of countries,
use **chcp 65001**.

No need to switch code page
per country.



Standard input & output

The real appearance of printf function

- Source code : <https://github.com/lattera/glibc/tree/master/stdio-common>

Standard Output (printf)

1. Syntax of **printf()** : Grammar and Structure of language
- #include <stdio.h>
int printf(const char *format, ...);
- The first argument (format) is a string containing text and format specifiers
 - The ellipsis (...) represents a variable number of arguments, which are inserted into the format specifier

Specifier	Data Type	Example
%d or %i	Integer (decimal)	printf("%d", 10); → 10
%f	Floating-point (decimal)	printf("%f", 3.14); → 3.140000
%.nf	Floating-point (n decimal places)	printf("%.2f", 3.14159); → 3.14
%c	Single character	printf("%c", 'A'); → A
%s	String	printf("%s", "Hello"); → Hello
%x or %X	Hexadecimal integer	printf("%x", 255); → ff
%o	Octal integer	printf("%o", 10); → 12
%p	Pointer (memory address)	printf("%p", ptr);
%%	Literal % symbol	printf("%%"); → %

Standard Output (printf)

Field width and precision

When printing using printf() , you can specify the size of the field in which data is printed .

output statement	output result	explanation										
printf("%10d", 123);	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td>2</td><td>3</td></tr></table>								1	2	3	Width 10, Right aligned
							1	2	3			
printf("%-10d", 123);	<table><tr><td>1</td><td>2</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	2	3								Width 10, Left aligned
1	2	3										

output statement	output result	explanation										
printf("%f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>8</td><td></td><td></td></tr></table>	1	.	2	3	4	5	6	8			6 decimal places
1	.	2	3	4	5	6	8					
printf("%10.3f", 1.23456789);	<table><tr><td></td><td></td><td></td><td></td><td></td><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td></tr></table>						1	.	2	3	5	3 decimal places
					1	.	2	3	5			
printf("%-10.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						3 decimal places
1	.	2	3	5								
printf("%.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						3 decimal places
1	.	2	3	5								

```
component_name : TEZ_CLIENT      recovery_enabled : true
component_name : WEBHCAT_SERVER  recovery_enabled : true
component_name : YARN_CLIENT     recovery_enabled : true
component_name : ZKFC            recovery_enabled : true
component_name : ZOOKEEPER_CLIENT recovery_enabled : true
component_name : ZOOKEEPER_SERVER recovery_enabled : true
```



```
component_name : TEZ_CLIENT      recovery_enabled : true
component_name : WEBHCAT_SERVER  recovery_enabled : true
component_name : YARN_CLIENT     recovery_enabled : true
component_name : ZKFC            recovery_enabled : true
component_name : ZOOKEEPER_CLIENT recovery_enabled : true
component_name : ZOOKEEPER_SERVER recovery_enabled : true
```

Standard Output (printf)

3. Format of printf()

- Width and Alignment

You can specify a **minimum width** for the output using numbers.

Default is right-aligned; to left-align, use -

```
printf("%10d\n", 123); // Right-aligned, width 10  
printf("%-10d\n", 123); // Left-aligned, width 10
```

Practice

- Precision for Floating-Point Numbers

```
printf("%.2f\n", 3.14159); // Prints with 2 decimal places
```

Practice

- Padding with Zeros

```
printf("%05d\n", 42); // Pads with zeros up to 5 digits
```

Practice

Standard Output (printf)

4. Using Escape Sequences of printf()

- printf supports escape sequences to control output formatting

Escape Sequence	Meaning	Example Output
<code>\n</code>	Newline	<code>printf("Hello\nWorld");</code> → Hello World
<code>\t</code>	Tab	<code>printf("Hello\tWorld");</code> → Hello World
<code>\\</code>	Backslash	<code>printf("C:\\Program Files\\");</code> → C:\Program Files\
<code>\"</code>	Double Quote	<code>printf("\"Hello\"");</code> → "Hello"

Practice

Standard Output (printf)

control character	name	meaning
<code>\0</code>	null character	
<code>\a</code>	warning(bell)	A "beep" warning sound occurs
<code>\b</code>	backspace	Moves the cursor one character back from its current position.
<code>\t</code>	horizontal tab	Moves the cursor position to the next tab position set on the current line.
<code>\n</code>	newline	Moves the cursor to the beginning of the next line.
<code>\v</code>	vertical tab	Move the cursor to the next set vertical tab position
<code>\f</code>	form feed	It is mainly used in printers to force the printer to turn to the next page.
<code>\r</code>	carriage return	Moves the cursor to the beginning of the current line.
<code>\"</code>	double quotation marks	The original double quotes themselves
<code>\'</code>	single quote	The original single quote itself
<code>\\</code>	backslash	The original backslash itself

Standard Output (printf)

5. Printing Multiple Values

- You can print multiple values in a single printf call by passing multiple

Practice

```
int age = 25;
float pi = 3.14;
printf("Age: %d, Pi: %.2f\n", age, pi);
```

5. Return Value of printf

- printf returns the number of characters printed (excluding \0)

Practice

```
int count = printf("Hello");
printf("\nCharacters printed: %d\n", count);
```

Standard Output (puts)

1. Syntax of **puts()**

```
#include <stdio.h>  
int puts(const char *str);
```

- puts prints a string (str) to the console and automatically appends a newline (\n) at the end.
- It is simpler and safer than printf("%s\n", str); because it does not require format specifiers.
- It returns a non-negative integer on success and EOF (-1) on failure.

```
#include <stdio.h>
```

```
int main() {  
    puts("Hello, World!");  
    return 0;  
}
```

Practice

Standard Output (putchar)

1. Syntax of **putchar()**

```
#include <stdio.h>
int putchar(int ch);
```

- putchar prints a single character (ch) to the console.
- It is simpler and safer than printf("%s\n", str); because it does not require format specifiers.
- It returns a non-negative integer on success and EOF (-1) on failure.

```
#include <stdio.h>

int main() {
    putchar('A');
    putchar('\n'); // Manually adding a newline
    return 0;
}
```

Practice

Standard Input

Standard input reads data from an input device, typically the keyboard.
It uses functions like `scanf()`, `getchar()`, and `fgets()` to read user input

[Keyboard typing]



[OS input buffer (stdin)]

↓ (flush when Enter pressed)

[Program function: `scanf/getchar/fgets...`]



[Stored in variable]

User input: hello↵

stdin buffer: 'h' 'e' 'l' 'l' 'o' '\n' '\0'

Standard Input (scanf)

1. Syntax of **scanf()**

```
#include <stdio.h>
int scanf(const char *format, ...);
```

- scanf reads formatted input from stdin (usually the keyboard).
- It requires format specifiers to determine the type of input.
- It stops reading when encountering whitespace (spaces, tabs, newlines, etc.).
- it remains '\n' in buffer. To avoid → getchar();

```
#include <stdio.h>

int main() {
    int age;
    float height;
    printf("Enter your age and height: ");
    scanf("%d %f", &age, &height);
    printf("You are %d years old and %.2f meters tall.\n", age, height);
    return 0;
}
```

Practice

Standard Input (scanf)

2. Key Characteristics of **scanf()**

- Can read multiple values at once.
- Requires the address-of operator (&) for non-string variables.
- Stops reading at the first whitespace character (space, tab, or newline).
- Can cause buffer issues if not used carefully (e.g., failing to handle newline characters properly).

If the user types hello↵:

- The OS puts hello↵ in the stdin buffer.
 - scanf("%s", str) reads hello and stops at the newline.
- The ↵ stays in the buffer.

Standard Input (scanf)

2. Key Characteristics of **scanf()**

```
int a, b;  
scanf("%d%d", &a, &b); // works the same as "%d %d"
```

Input: 1 2 (multiple spaces or tabs or newline)

- First %d → 1
- Whitespace skipped automatically
- Second %d → 2

Standard Input (getchar)

1. Syntax of **getchar()**

```
#include <stdio.h>
int getchar(void);
```

- getchar reads a single character from stdin.
- It includes whitespace characters like spaces and newlines.
- Returns the character as an unsigned char (cast to int) or EOF on error.

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: %c\n", ch);
    return 0;
}
```

Practice

Standard Input (Handling Newline (`\n`) Issues)

problem: Buffer Retains `\n`

If `getchar` is used after `scanf`, it may read the leftover newline (`\n`).

* Find the problem below

```
#include <stdio.h>

int main() {
    int num;
    char ch;

    printf("Enter a number: ");
    scanf("%d", &num); // Reads a number

    printf("Enter a character: ");
    ch = getchar(); // Problem: This reads the newline ('\n') from the buffer!

    printf("Number: %d, Character: %c\n", num, ch);
    return 0;
}
```

Find the problems

Standard Input (fgets)

1. Syntax of **fgets()**

```
#include <stdio.h>
char *fgets(char *str, int n, FILE *stream);
```

- fgets reads a whole line from the input (up to n-1 characters).
- It includes spaces and stops at a newline (\n).
- It prevents buffer overflow by specifying the maximum number of characters.
- It includes '\n' → hello\n\0

```
#include <stdio.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);
    printf("Hello, %s", name);
    return 0;
}
```

Practice

Standard Input (fgets)

2. Key Characteristics of **fgets()**

- Reads a full line, including spaces.
- Stops when newline ($\backslash n$) or buffer limit ($n-1$ characters) is reached.
- Unlike scanf, it does not skip spaces.
- Adds a newline character ($\backslash n$) if the user presses Enter.

Standard Input

Comparison of scanf, getchar, fgets

-

Feature	scanf	getchar	fgets
Reads	Formatted input (integers, floats, strings, etc.)	Single character	Whole line (string)
Stops at	Whitespace (space, tab, newline)	Single character (including spaces)	Newline (<code>\n</code>) or max buffer size
Handles whitespace	Ignores leading spaces	Reads spaces & newlines	Includes spaces, retains newline (<code>\n</code>)
Best for	Numeric input or formatted data	Single character input	Full-line string input
Risk of buffer overflow?	Yes (if not handled properly)	No	No (safe with buffer size)
Newline handling	Left in buffer (needs clearing)	Consumed as input	Stored in string (needs removal if unwanted)

Standard Input

When to use which?

-

Scenario	Best Choice
Reading a single integer or float	<code>scanf</code>
Reading a single character	<code>getchar</code>
Reading an entire line of text (including spaces)	<code>fgets</code>
Reading formatted input (e.g., "Name Age Height")	<code>scanf</code>
Avoiding buffer overflow issues when reading strings	<code>fgets</code>

Standard Input

How each function handles Enter (`\n`)?

-

Function	Reads <code>\n</code> ?	When does it capture <code>\n</code> ?	How to handle it?
<code>scanf("%d")</code>	✗ No	Skips whitespace, including <code>\n</code>	No need
<code>scanf("%c")</code>	✓ Yes	Captures leftover <code>\n</code> if input before it doesn't consume it	Use <code>" %c"</code> to skip whitespace
<code>getchar()</code>	✓ Yes	Always reads <code>\n</code> if it's in the buffer	Use multiple <code>getchar()</code> calls if needed
<code>fgets()</code>	✓ Yes	Always stores <code>\n</code> in the string (if space allows)	Remove with <code>strcspn()</code>

Problem solving with stdio.h

1. Determine what the input is
 - scanf, fgets, getchar, sscanf

User scenario

- > 1
- > 1 3.14
- > name age email
- > name age email introduction
- > name,age,email,introduction
- > Name:abc, Age:20, Email:abc@email.com, Introduction: I am a boy
- > word sentence number etc

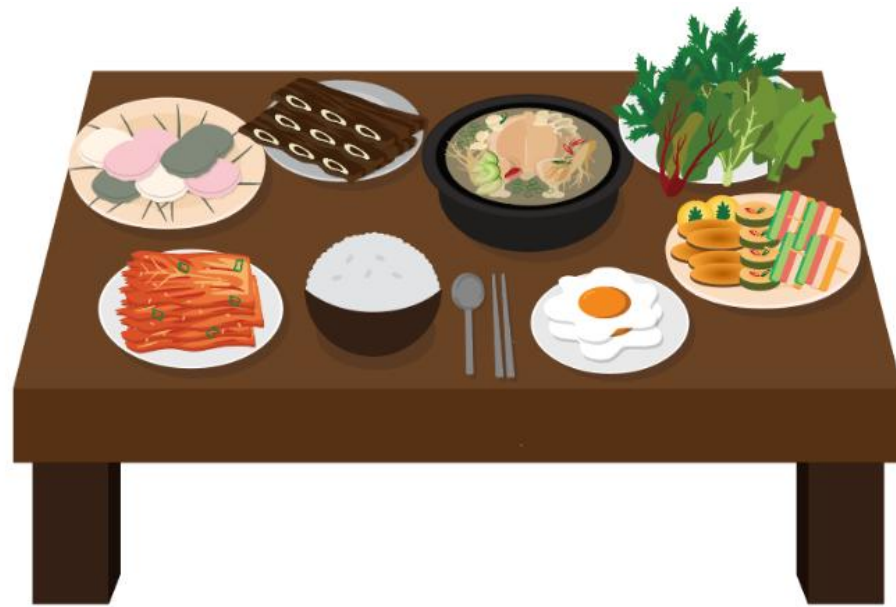
See you next week!

DO NOT miss the classes



변수 (Variable) & Data types

- 변수: 데이터를 저장할 수 있는 메모리 공간 (그릇)
 - 그릇에는 밥, 반찬, 물 등을 담을 수 있음.



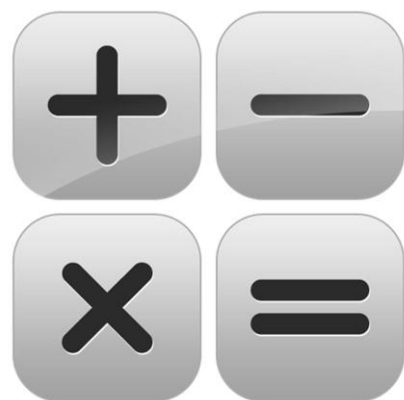
- 변수 생성 규칙
 - 예약어(키워드)는 사용할 수 없음 (예: for, if, else 등)
 - 공백(띄어쓰기)을 포함할 수 없음
 - 첫 글자는 영문자 또는 언더스코어(_)만 가능 (숫자로 시작 ✕)
 - 언더스코어(_)를 제외한 특수문자는 사용할 수 없음
 - 대소문자를 구분함

연산자 (Operators)

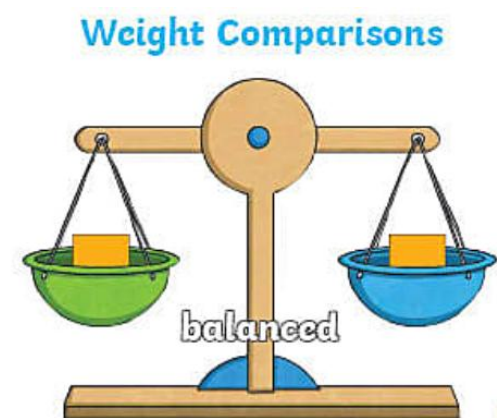
프로그램에서 연산자(operator)란, 값(데이터)에 어떤 동작을 수행하도록 지시하는 기호나 단어를 의미합니다.

[대표적인 연산자 종류]

1. 산술 연산자 (계산할 때)



2. 비교 연산자 (값 비교)



1. 논리 연산자 (조건 연결)

Input		Output
A	B	
0	0	0
0	1	0
1	0	0
1	1	1

AND

Input		Output
A	B	
0	0	0
0	1	1
1	0	1
1	1	1

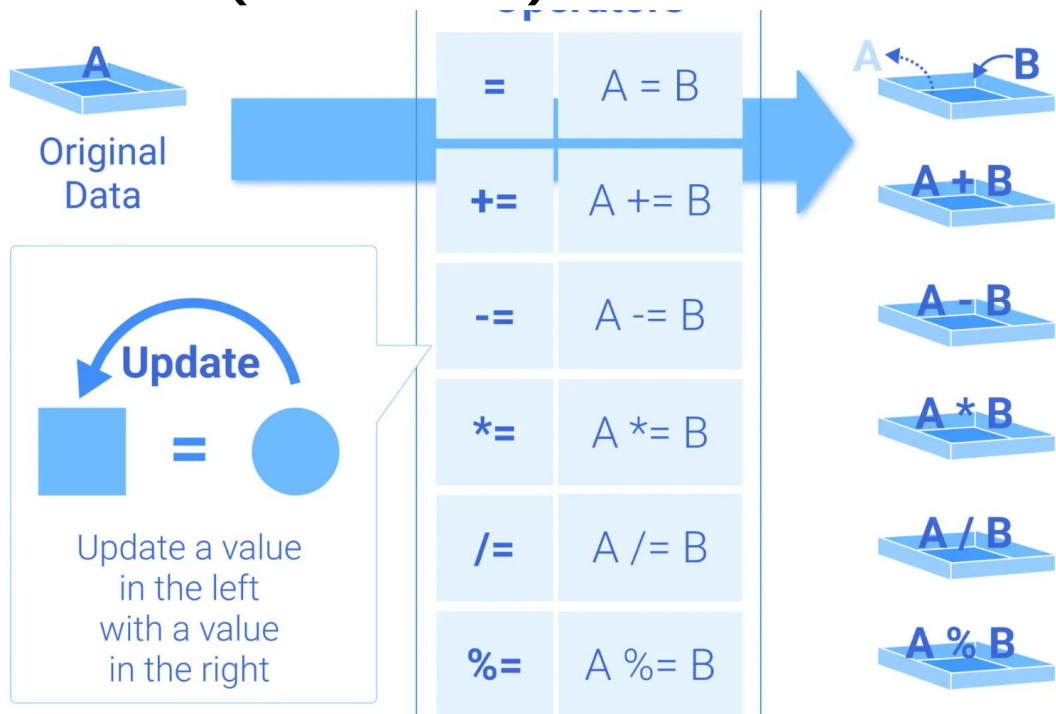
OR

Input	Output
A	
0	1
1	0

NOT

AND (&&) → 둘 다 만족해야 true
OR (||) → 하나만 만족해도 true
NOT (!) → 반대로 바꿈

4. 대입 연산자 (값 저장)



Variables & 데이터 타입 (Data types)

- Data types: 메모리 공간을 효율적으로 사용하기 위해서는, 적절한 형태와 크기의 데이터 타입을 사용해야 한다.

데이터 타입	설명	크기	범위	예시
int	정수를 저장하는 데이터 타입	4	-2,147,483,648 ~ 2,147,483,647	int num = 10;
long	더 큰 정수를 저장하는 데이터 타입	8	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807	long num = 1000000L;
unsigned int	양수만 저장하는 정수형	4	0 ~ 4,294,967,295	unsigned int num = 10U;
unsigned long	양수만 저장하는 큰 정수형	8	0 ~ 18,446,744,073,709,551,615	unsigned long num = 1000000UL;
char	하나의 문자를 저장하는 데이터 타입	1	-128 ~ 127 (signed) 또는 0 ~ 255 (unsigned)	char letter = 'A';
unsigned char	양수 문자 값만 저장 (0~255)	1	0 ~ 255	unsigned char letter = 65U;
float	단정도 실수형 (소수)	4	$\pm 1.5 \times 10^{-45} \sim \pm 3.4 \times 10^{38}$	float num = 3.14f;
double	배정도 실수형 (float보다 더 정밀함)	8	$\pm 5.0 \times 10^{-324} \sim \pm 1.7 \times 10^{308}$	double num = 3.141592;
void	데이터 없음 또는 함수 반환값 없음	없음	없음	void function() {}

유니코드 (Unicode)

C 언어는 원래 ASCII(1바이트 문자)를 기반으로 설계되었지만,
현대의 프로그램은 여러 언어의 문자를 처리하기 위해 **유니코드 지원**이 필요하다.
유니코드는 **UTF-8, UTF-16, UTF-32, CP949**(windows) 와 같은 다양한 인코딩 방식으로 사용된다.

유니코드란 모든 언어의 모든 문자에 고유한 번호를 부여하는 범용 문자 체계.

예) 'A' → U+0041 (단순 숫자)

'가' → U+AC00

'あ' → U+3042

Unicode defines the meaning of characters but not how they are stored in memory or files.

Range: U+0000 ~ U+10FFFF → a total of 1,114,112 possible code points.

Within this range, Unicode is designed to represent all the world's scripts, emojis, symbols, and even ancient characters.

Method of expression using UTF-8:

- U+0000 ~ U+007F → 1 byte
- U+0080 ~ U+07FF → 2 bytes
- U+0800 ~ U+FFFF → 3 bytes
- U+10000 ~ U+10FFFF → 4 bytes

C에서의 유니코드 인코딩

인코딩(Encoding): 유니코드 문자를 바이트로 변환하여 저장하거나 전송할 수 있게 하는 방법

유니코드 문자의 표현 방식:

- **UTF-8** (가변 길이 (1~4바이트)) → 가장 널리 사용됨 (Linux/macOS 기본)
 - ASCII 문자: 1 byte
 - 유럽 문자: 2 bytes
 - 한국어/중국어 등: 3 ~ 4 bytes
- **1바이트**: 기본 ASCII 문자 (예: A, B, C, 0~9) → 0xxxxxxx
- **2바이트**: 유럽 언어, 한글 자모 (예: é, ñ, ü, 한) → 110xxxxx 10xxxxxx
- **3바이트**: 대부분의 한글, 한자, 일부 이모지 → 1110xxxx 10xxxxxx 10xxxxxx
- **4바이트**: 확장 문자 및 이모지 (예: 😊, 🍷) → 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

표준 입출력

표준 출력 (printf)

```
#include <stdio.h>
int printf(const char *format, ...);
```

1. printf() 함수의 구문: 언어의 문법 및 구조

- 첫 번째 인자(format)는 텍스트와 형식 지정자를 포함하는 문자열입니다.
- 줄임표(...)는 가변적인 개수의 인자를 나타내며, 이 인자들은 형식 지정자 앞에 삽입됩니다.

Specifier	Data Type	Example
%d or %i	Integer (decimal)	printf("%d", 10); → 10
%f	Floating-point (decimal)	printf("%f", 3.14); → 3.140000
%.nf	Floating-point (n decimal places)	printf("%.2f", 3.14159); → 3.14
%c	Single character	printf("%c", 'A'); → A
%s	String	printf("%s", "Hello"); → Hello
%x or %X	Hexadecimal integer	printf("%x", 255); → ff
%o	Octal integer	printf("%o", 10); → 12
%p	Pointer (memory address)	printf("%p", ptr);
%%	Literal % symbol	printf("%%"); → %

표준 출력 (printf)

필드 폭 및 정밀도

printf() 함수를 사용하여 출력할 때, 데이터가 출력될 필드의 크기를 지정할 수 있습니다.

output statement	output result	explanation										
printf("%10d", 123);	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>1</td><td>2</td><td>3</td></tr></table>								1	2	3	Width 10, Right aligned
							1	2	3			
printf("%-10d", 123);	<table><tr><td>1</td><td>2</td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	2	3								Width 10, Left aligned
1	2	3										

output statement	output result	explanation										
printf("%f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>8</td><td></td><td></td></tr></table>	1	.	2	3	4	5	6	8			6 decimal places
1	.	2	3	4	5	6	8					
printf("%10.3f", 1.23456789);	<table><tr><td></td><td></td><td></td><td></td><td></td><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td></tr></table>						1	.	2	3	5	3 decimal places
					1	.	2	3	5			
printf("%-10.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						3 decimal places
1	.	2	3	5								
printf("%.3f", 1.23456789);	<table><tr><td>1</td><td>.</td><td>2</td><td>3</td><td>5</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1	.	2	3	5						3 decimal places
1	.	2	3	5								

component_name : TEZ_CLIENT

recovery_enabled : true

component_name : WEBHCAT_SERVER

recovery_enabled : true

component_name : YARN_CLIENT

recovery_enabled : true

component_name : ZKFC

recovery_enabled : true

component_name : ZOOKEEPER_CLIENT

recovery_enabled : true

component_name : ZOOKEEPER_SERVER

recovery_enabled : true



component_name : TEZ_CLIENT

recovery_enabled : true

component_name : WEBHCAT_SERVER

recovery_enabled : true

component_name : YARN_CLIENT

recovery_enabled : true

component_name : ZKFC

recovery_enabled : true

component_name : ZOOKEEPER_CLIENT

recovery_enabled : true

component_name : ZOOKEEPER_SERVER

recovery_enabled : true



3. printf() 함수의 형식

- 너비 및 정렬숫자를 사용하여 출력의 최소 너비를 지정할 수 있습니다.
- 기본값은 오른쪽 정렬이며, 왼쪽 정렬하려면 -를 사용합니다.

```
printf("%10d\n", 123); // Right-aligned, width 10  
printf("%-10d\n", 123); // Left-aligned, width 10
```

Practice

- 부동소수점 숫자의 정밀도

```
printf("%.2f\n", 3.14159); // Prints with 2 decimal places
```

Practice

- 0으로 채우기

```
printf("%05d\n", 42); // Pads with zeros up to 5 digits
```

Practice

표준 출력 (printf)

4. printf() 함수의 이스케이프 시퀀스 사용

printf 함수는 출력 형식을 제어하기 위해 이스케이프 시퀀스를 지원합니다.

Escape Sequence	Meaning	Example Output
<code>\n</code>	Newline	<code>printf("Hello\nWorld");</code> → <code>Hello</code> <code>World</code>
<code>\t</code>	Tab	<code>printf("Hello\tWorld");</code> → <code>Hello World</code>
<code>\\</code>	Backslash	<code>printf("C:\\Program Files\\");</code> → <code>C:\Program Files\</code>
<code>\"</code>	Double Quote	<code>printf("\"Hello\"");</code> → <code>"Hello"</code>

Practice

표준 출력 (printf)

제어 문자	이름	의미
\0	널 문자	
\a	경고(벨)	"삐" 경고음이 울립니다.
\b	백스페이스	커서를 현재 위치에서 한 문자 뒤로 이동합니다.
\t	가로 탭	커서를 현재 줄에 설정된 다음 탭 위치로 이동합니다.
\n	줄 바꿈	커서를 다음 줄의 시작 부분으로 이동합니다.
\v	세로 탭	커서를 다음 설정된 세로 탭 위치로 이동합니다.
\f	용지 공급	주로 프린터에서 프린터가 다음 페이지로 넘어가도록 강제하는 데 사용됩니다.
\r	캐리지 리턴	커서를 현재 줄의 시작 부분으로 이동합니다.
\"	큰따옴표	원래의 큰따옴표 자체
\'	작은따옴표	원래의 작은따옴표 자체
\\	백슬래시	원래의 백슬래시 자체

표준 출력 (printf)

5. 여러 값 출력하기

- printf 함수 호출 한 번에 여러 값을 출력하려면 여러 개의 매개변수를 전달하면 됩니다.

```
int age = 25;
float pi = 3.14;
printf("Age: %d, Pi: %.2f\n", age, pi);
```

Practice

5. printf 함수의 반환 값

- printf 함수는 출력된 문자 수(\0 제외)를 반환합니다.

```
int count = printf("Hello");
printf("\nCharacters printed: %d\n", count);
```

Practice

표준 출력 (puts)

1. puts() 함수의 구문

```
#include <stdio.h>
int puts(const char *str);
```

- puts 함수는 문자열(str)을 콘솔에 출력하고 자동으로 줄 바꿈 문자(Wn)를 추가합니다.
- printf("%sWn", str);)보다 간단하고 안전합니다. 형식 지정자가 필요 없기 때문입니다.
- 성공 시 음수가 아닌 정수를 반환하고 실패 시 EOF(-1)를 반환합니다.

```
#include <stdio.h>
```

```
int main() {
```

```
    puts("Hello, World!");
```

```
    return 0;
```

```
}
```

Practice

표준 출력 (putchar)

1. putchar() 함수의 구문

```
#include <stdio.h>
int putchar(int ch);
```

- putchar 함수는 콘솔에 단일 문자(ch)를 출력합니다.
- printf("%s\n", str);보다 간단하고 안전합니다. 형식 지정자가 필요하지 않기 때문입니다.
- 성공 시 음수가 아닌 정수를 반환하고, 실패 시 EOF(-1)를 반환합니다.

```
#include <stdio.h>

int main() {
    putchar('A');
    putchar('\n'); // Manually adding a newline
    return 0;
}
```

Practice

표준 입력

표준 입력은 입력 장치, 일반적으로 키보드에서 데이터를 읽습니다.

사용자 입력을 읽기 위해 scanf(), getchar(), fgets()와 같은 함수를 사용합니다.

[키보드 입력]



[OS 입력 버퍼(stdin)]



[프로그램 함수: scanf/getchar/fgets 등]



[변수에 저장됨]

사용자 입력: hello↵

입력 버퍼 : 'h' 'e' 'l' 'l' 'o' '\n' '\0'

표준 입력 (scanf)

1. scanf() 함수의 구문

```
#include <stdio.h>
```

```
int scanf(const char *format, ...);
```

- scanf는 표준 입력(일반적으로 키보드)에서 형식화된 입력을 읽습니다.
- 입력 유형을 결정하기 위해 형식 지정자가 필요합니다.
- 공백 문자(공백, 탭, 줄 바꿈 등)를 만나면 읽기를 중지합니다.
- 버퍼에 '\n'이 남아 있습니다. 이를 방지하려면 → getchar()를 사용하십시오.

```
#include <stdio.h>

int main() {
    int age;
    float height;
    printf("Enter your age and height: ");
    scanf("%d %f", &age, &height);
    printf("You are %d years old and %.2f meters tall.\n", age, height);
    return 0;
}
```

Practice

Standard Input (scanf)

2. scanf() 함수의 주요 특징

- 여러 값을 한 번에 읽을 수 있습니다.
- 문자열이 아닌 변수에는 주소 연산자(&)가 필요합니다.
- 첫 번째 공백 문자(공백, 탭 또는 줄 바꿈)에서 읽기를 멈춥니다.
- 주의해서 사용하지 않으면 버퍼 문제가 발생할 수 있습니다
(예: 줄 바꿈 문자를 제대로 처리하지 못하는 경우).

사용자가 hello↵를 입력하면 다음과 같습니다.

- 운영체제는 표준 입력 버퍼에 hello\n을 넣습니다.
- scanf("%s", str)는 hello를 읽고 줄 바꿈 문자에서 멈춥니다.
- \n은 버퍼에 그대로 남아 있습니다.

표준 입력 (scanf)

2. scanf() 함수의 주요 특징

```
int a, b;  
scanf("%d%d", &a, &b); // "%d %d"와 동일하게 작동합니다.
```

입력: 1 2 (여러 개의 공백, 탭 또는 줄 바꿈 포함)

- 첫 번째 %d → 1
- 공백은 자동으로 건너뜁니다.
- 두 번째 %d → 2

표준 입력 (getchar)

1. getchar() 함수의 구문

```
#include <stdio.h>
int getchar(void);
```

- getchar 함수는 표준 입력(stdin)에서 문자 하나를 읽습니다.
- 공백이나 줄 바꿈과 같은 공백 문자도 포함됩니다.
- 읽은 문자를 부호 없는 문자(unsigned char, int로 형변환 필요)로

반환하거나,

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: %c\n", ch);
    return 0;
}
```

환합니다.

Practice

표준 입력 (Newline (\n) 문제 해결방법)

문제: 버퍼에 줄 바꿈 문자(\n)가 남아 있음

scanf 다음에 getchar를 사용하면 남은 줄 바꿈 문자(\n)를 읽을 수 있습니다.

*아래에서 문제를 확인하세요

```
#include <stdio.h>

int main() {
    int num;
    char ch;

    printf("Enter a number: ");
    scanf("%d", &num); // Reads a number

    printf("Enter a character: ");
    ch = getchar(); // Problem: This reads the newline ('\n') from the buffer!

    printf("Number: %d, Character: %c\n", num, ch);
    return 0;
}
```

Find the problems

표준 입력 (fgets)

1. fgets() 함수의 구문

```
#include <stdio.h>
char *fgets(char *str, int n, FILE *stream);
```

- fgets는 입력에서 한 줄 전체(최대 n-1개 문자)를 읽습니다.
- 공백을 포함하며 줄 바꿈 문자(\n)에서 멈춥니다.
- 최대 문자 수를 지정하여 버퍼 오버플로(overflow)를 방지합니다.
- '\n'도 포함합니다(예: hello\n\0).

```
#include <stdio.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);
    printf("Hello, %s", name);
    return 0;
}
```

Practice

표준 입력 (fgets)

2. fgets() 함수의 주요 특징

- 공백을 포함하여 한 줄 전체를 읽습니다.
- 줄 바꿈 문자(\n) 또는 버퍼 제한(n-1 문자)에 도달하면 실행을 중지합니다.
- scanf 함수와 달리 공백을 건너뛰지 않습니다.
- 사용자가 Enter 키를 누르면 줄 바꿈 문자(\n)를 추가합니다.

표준 입력 Summary

scanf, getchar, fgets 비교표

-

기능	scanf	getchar	fgets
읽기	형식화된 입력(정수, 부동 소수점, 문자열 등)	단일 문자	전체 줄(문자열)
종지 위치	공백(공백, 탭, 줄 바꿈)	단일 문자(공백 포함)	줄 바꿈(\n) 또는 최대 버퍼 크기
공백 처리	선행 공백 무시	공백 및 줄 바꿈 읽기	공백 포함, 줄 바꿈(\n) 유지
최적 사용 대상	숫자 입력 또는 형식화된 데이터	단일 문자 입력	전체 줄 문자열 입력
버퍼 오버플로 위험?	예(적절하게 처리되지 않을 경우)	아니요	아니요(버퍼 크기가 적절하면 안전)
줄 바꿈 처리	버퍼에 남아 있음(비워야 함)	입력으로 사용	문자열에 저장됨 (원치 않을 경우 제거해야 함)

표준 입력

어떤 상황에 어떤 함수를 사용하는가?

시나리오	최적의 선택
단일 정수 또는 부동 소수점 읽기	<code>scanf</code>
단일 문자 읽기	<code>getchar</code>
전체 텍스트 줄 읽기 (공백 포함)	<code>fgets</code>
형식화된 입력 읽기 (예: "이름 나이 키")	<code>scanf</code>
문자열을 읽을 때 버퍼 오버플로 문제 방지	<code>fgets</code>

표준 입력

각 함수는 어떻게 ‘\n’ 문제를 해결하는가?

-

함수	\n을 읽나요?	언제 \n을 캡처하나요?	어떻게 처리하나요?
<code>scanf("%d")</code>	X 아니요	\n을 포함한 공백을 건너뜁니다.	필요 없음
<code>scanf("%c")</code>	✓ 예	이전 입력에서 \n을 소비하지 않으면 남은 \n을 캡처합니다.	공백을 건너뛰려면 "%c"를 사용하세요.
<code>getchar()</code>	✓ 예	버퍼에 \n이 있으면 항상 읽습니다.	필요한 경우 여러 번 <code>getchar()</code> 를 호출하세요.
<code>fgets()</code>	✓ 예	(공간이 허용하는 경우) 항상 \n을 문자열에 저장합니다.	<code>strcspn()</code> 으로 제거합니다

시나리오

1. 어떤 함수를 사용하나요?
 - scanf, fgets, getchar, sscanf

사용자 시나리오

- > 1
- > 1 3.14
- > name age email
- > name age email introduction
- > name,age,email,introduction
- > Name:abc, Age:20, Email:abc@email.com, Introduction: I am a boy
- > word sentence number etc